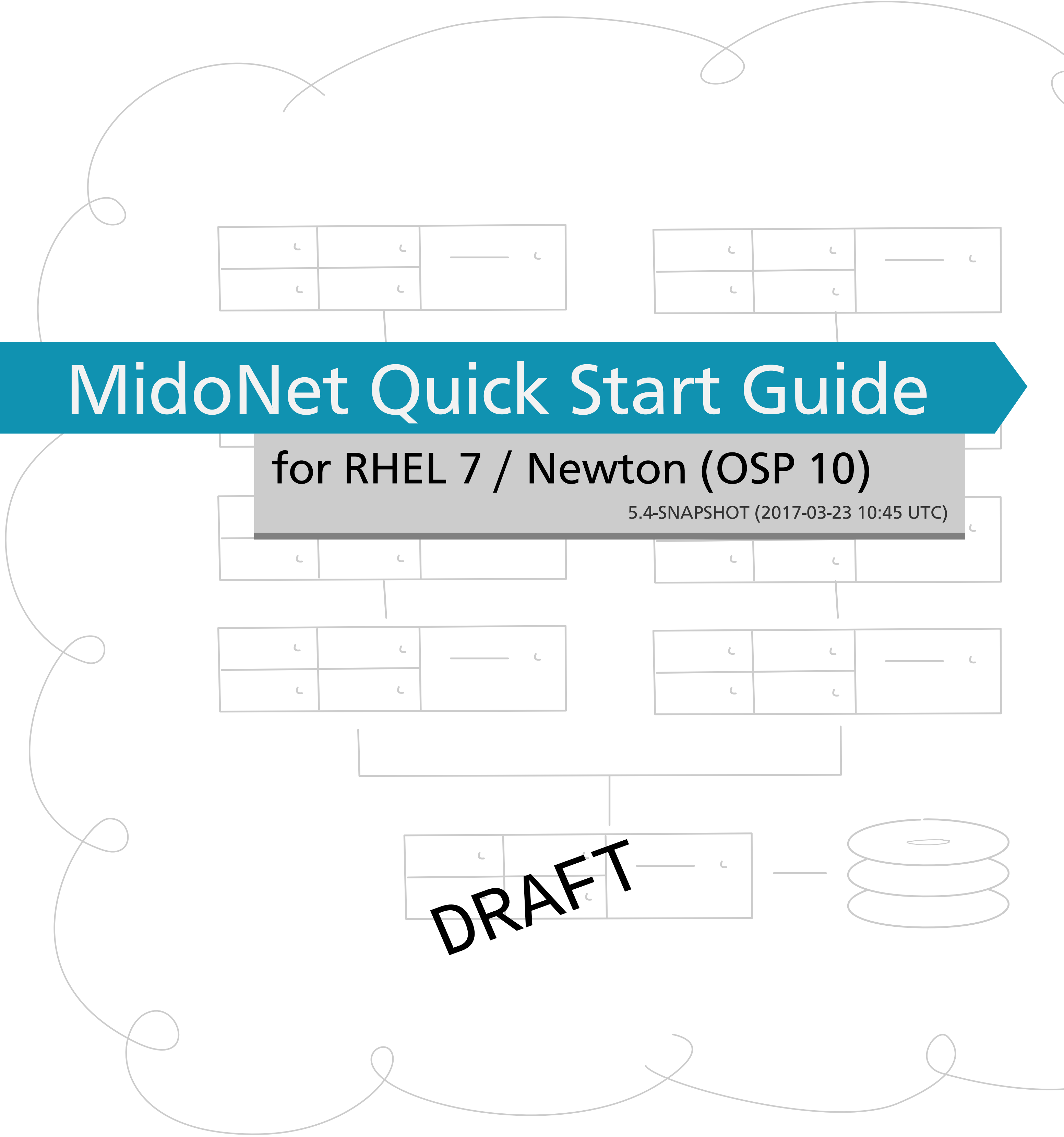

MidoNet Quick Start Guide

for RHEL 7 / Newton (OSP 10)

5.4-SNAPSHOT (2017-03-23 10:45 UTC)

DRAFT

midonet

docs.midonet.org

MidoNet Quick Start Guide for RHEL 7 / Newton (OSP 10)

5.4-SNAPSHOT (2017-03-23 10:45 UTC)
Copyright © 2017 Midokura SARL All rights reserved.

MidoNet is a network virtualization software for Infrastructure-as-a-Service (IaaS) clouds.

It decouples your IaaS cloud from your network hardware, creating an intelligent software abstraction layer between your end hosts and your physical network.

This guide walks through the minimum installation and configuration steps neccessary to use MidoNet with OpenStack.

Caution

This document is a DRAFT. It may be MISSING relevant information or contain UNTESTED information. Use it at your own risk.

Note

Please consult the MidoNet Mailing Lists or Chat if you need assistance.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Table of Contents

Preface iv
  Conventions iv
1. Architecture 1
  Hosts and Services 2
2. Basic Environment Configuration 4
  Networking Configuration 4
  SELinux Configuration 4
  Repository Configuration 4
3. OpenStack Installation 6
  Identity Service (Keystone) 6
  Compute Services (Nova) 6
  Networking Services (Neutron) 10
4. MidoNet Installation 17
  NSDB Nodes 17
  Controller Node 20
  Midolman Installation 22
  MidoNet Host Registration 23
5. Initial Network Configuration 25
6. Edge Router Setup 26
7. BGP Uplink Configuration 28
8. Further Steps 30

Preface

Conventions

The MidoNet documentation uses several typesetting conventions.

Notices

Notices take these forms:

Note

A handy tip or reminder.

Important

Something you must be aware of before proceeding.

Warning

Critical information about the risk of data loss or security issues.

Command prompts

$ prompt

Any user, including the root user, can run commands that are prefixed with the $ prompt.

# prompt

The root user must run commands that are prefixed with the # prompt. You can also prefix these commands with the **sudo** command, if available, to run them.

1. Architecture

Table of Contents

Hosts and Services 2

This guide assumes the following example system architecture.

OpenStack Controller Node:

- Controller Node (**controller**)

Compute Node:

- Compute Node (**compute1**)

Since MidoNet is a distributed system, it does not have the concept of a Network Node as being used with the default OpenStack networking plugin. Instead it uses two or more Gateway Nodes that utilize Quagga to provide connectivity to external networks via the Border Gateway Protocol (BGP).

- Gateway Node 1 (**gateway1**)
- Gateway Node 2 (**gateway2**)

Three or more hosts are being used for the MidoNet Network State Database (NSDB) cluster which utilizes ZooKeeper and Cassandra to store virtual network topology and connection state information:

- NSDB Node 1 (**nsdb1**)
- NSDB Node 2 (**nsdb2**)
- NSDB Node 3 (**nsdb3**)

Important

Ideally, both the ZooKeeper transaction log and Cassandra data files need their own dedicated disks, with additional disks for other services on the host. However, for small POCs and small deployments, it is ok to share the Cassandra disk with other services and just leave the ZooKeeper transaction log on its own.

The *MidoNet Agent (Midolman)* has to be installed on all nodes where traffic enters or leaves the virtual topology. In this guide this are the **gateway1**, **gateway2** and **compute1** hosts.

The *Midonet Cluster* can be installed on a separate host, but this guide assumes it to be installed on the **controller** host.

The *Midonet Command Line Interface (CLI)* can be installed on any host that has connectivity to the MidoNet Cluster. This guide assumes it to be installed on the **controller** host.

The *Midonet Neutron Plugin* replaces the ML2 Plugin and has to be installed on the **controller**.

Hosts and Services

Controller Node (controller)

- General
  - Database (MariaDB)
  - Message Broker (RabbitMQ)
- OpenStack
  - Identity Service (Keystone)
  - Image Service (Glance)
  - Compute (Nova)
  - Networking (Neutron)
    - Neutron Server
  - Dashboard (Horizon)
- MidoNet
  - Cluster
  - CLI
  - Neutron Plugin

Compute Node (compute1)

- OpenStack
  - Compute (Nova)
  - Networking (Neutron)
- MidoNet
  - Agent (Midolman)

NSDB Nodes (nsdb1, nsdb2, nsdb3)

- Network State Database (NSDB)
  - Network Topology (ZooKeeper)
  - Network State Information (Cassandra)

Gateway Nodes (gateway1, gateway2)

- BGP Daemon (Quagga)

- MidoNet
  - Agent (Midolman)

2. Basic Environment Configuration

Table of Contents

Networking Configuration 4
SELinux Configuration 4
Repository Configuration 4

Networking Configuration

Important

All hostnames must be resolvable, either via DNS or locally.

This guide assumes that you follow the instructions in Host Networking of the OpenStack Documentation.

SELinux Configuration

Important

This guide assumes that SELinux (if installed) is either in `permissive` state or `disabled`.

To change the mode, execute the following command:

```
# setenforce Permissive
```

To permanently change the SELinux configuration, edit the `/etc/selinux/config` file accordingly:

```
SELINUX=permissive
```

Repository Configuration

Configure necessary software repositories and update installed packages.

1. **Enable Red Hat base repository**

   ```
   # subscription-manager repos --enable=rhel-7-server-rpms
   ```

2. **Enable Red Hat OSP repositories**

   ```
   # subscription-manager repos --enable=rhel-7-server-openstack-10-rpms
   # subscription-manager repos --enable=rhel-7-server-openstack-10-
   optools-rpms
   ```

3. **Enable DataStax repository**

   Create the `/etc/yum.repos.d/datastax.repo` file and edit it to contain the following:

   ```
   # DataStax (Apache Cassandra)
   ```

```
[datastax]
name = DataStax Repo for Apache Cassandra
baseurl = http://rpm.datastax.com/community
enabled = 1
gpgcheck = 1
gpgkey = https://rpm.datastax.com/rpm/repo_key
```

1. **Enable MidoNet repositories**

   Create the `/etc/yum.repos.d/midonet.repo` file and edit it to contain the following:

   ```
   [midonet]
   name=MidoNet
   baseurl=http://builds.midonet.org/midonet-5.4/stable/el7/
   enabled=1
   gpgcheck=1
   gpgkey=https://builds.midonet.org/midorepo.key

   [midonet-openstack-integration]
   name=MidoNet OpenStack Integration
   baseurl=http://builds.midonet.org/openstack-newton/stable/el7/
   enabled=1
   gpgcheck=1
   gpgkey=https://builds.midonet.org/midorepo.key

   [midonet-misc]
   name=MidoNet 3rd Party Tools and Libraries
   baseurl=http://builds.midonet.org/misc/stable/el7/
   enabled=1
   gpgcheck=1
   gpgkey=https://builds.midonet.org/midorepo.key
   ```

2. **Install available updates**

   ```
   # yum clean all
   # yum upgrade
   ```

3. **If necessary, reboot the system**

   ```
   # reboot
   ```

3. OpenStack Installation

Table of Contents

Identity Service (Keystone) 6
Compute Services (Nova) 6
Networking Services (Neutron) 10

Important

Follow the Installation Reference documentation, but **note the following differences**.

Identity Service (Keystone)

Important

Follow the Red Hat documentation’s Chapter 3. Install The Identity Service instructions, but **note the following additions**.

1. **Create MidoNet API Service**

   As Keystone admin, execute the following command:

   ```
   $ openstack service create --name midonet --description "MidoNet API
    Service" midonet
   ```

2. **Create MidoNet Administrative User**

   As Keystone admin, execute the following commands:

   ```
   $ openstack user create --domain default --password-prompt midonet
   $ openstack role add --project service --user midonet admin
   ```

Compute Services (Nova)

Important

Follow the Red Hat documentation’s Chapter 8. Install The Compute Service instructions, but **note the following differences**.

Controller Node

Important

Follow the Red Hat documentation’s 8.2. Install a Compute Node instructions, but **note the following differences and additions**.

1. **8.2.1. Install the Compute Service Packages**

   Do **not** apply as is.

   Instead, install only the following packages:

```
# yum install openstack-nova-api openstack-nova-conductor openstack-
nova-scheduler python-cinderclient
```

Note

The `openstack-nova-compute` package is going to be installed on the Compute Node instead.

2. **8.2.2. Create the Compute Service Database**

   Apply as is.

3. **8.2.3. Configure the Compute Service Database Connection**

   Apply as is.

4. **8.2.4. Create the Compute Service Identity Records**

   Apply as is.

5. **8.2.5. Configure Compute Service Authentication**

   Apply as is.

6. **8.2.6. Configure the Firewall to Allow Compute Service Traffic**

   Apply as is.

7. **8.2.7. Configure the Compute Service to Use SSL**

   Apply as is.

8. **8.2.8. Configure RabbitMQ Message Broker Settings for the Compute Service**

   Apply as is.

9. **8.2.9. Enable SSL Communication Between the Compute Service and the Message Broker**

   Apply as is.

10. **8.2.10. Configure Resource Overcommitment**

    Apply as is.

11. **8.2.11. Reserve Host Resources**

    Apply as is.

12. **8.2.12. Configure Compute Networking**

    Apply as is, except the following topics:

    a. **8.2.12.3. Configure the L2 Agent**

       Do **not** apply.

    b. **8.2.12.4. Configure Virtual Interface Plugging**

       Configure the generic VIF driver.

13. **8.2.13. Populate the Compute Service Database**

    Apply as is.

14. **8.2.14. Launch the Compute Services**

    a. **1. Starting the Message Bus Service**

       Do **not** apply. Only required on the Compute Node.

    b. **2. Starting the Libvirtd Service**

       Do **not** apply. Only required on the Compute Node.

    c. **3. Starting the API Service**

       Apply as is.

    d. **4. Starting the Scheduler**

       Apply as is.

    e. **5. Starting the Conductor**

       Apply as is.

    f. **6. Starting the Compute Service**

       Do **not** apply. Only required on the Compute Node.

Compute Node

Important

Follow the Red Hat documentation’s 8.2. Install a Compute Node instructions, but **note the following differences and additions**.

1. **8.2.1. Install the Compute Service Packages**

   Do **not** apply as is.

   Instead, install only the following packages:

   ```
   # yum install openstack-nova-compute openstack-utils
   ```

2. **8.2.2. Create the Compute Service Database**

   Do **not** apply. Has been done on the Controller Node.

3. **8.2.3. Configure the Compute Service Database Connection**

   Apply as is.

4. **8.2.4. Create the Compute Service Identity Records**

Do **not** apply. Has been done on the Controller Node.

1. **8.2.5. Configure Compute Service Authentication**

Apply as is.

1. **8.2.6. Configure the Firewall to Allow Compute Service Traffic**

   Apply as is.

2. **8.2.7. Configure the Compute Service to Use SSL**

   Apply as is.

3. **8.2.8. Configure RabbitMQ Message Broker Settings for the Compute Service**

   Apply as is.

4. **8.2.9. Enable SSL Communication Between the Compute Service and the Message Broker**

   Apply as is.

5. **8.2.10. Configure Resource Overcommitment**

   Apply as is.

6. **8.2.11. Reserve Host Resources**

   Apply as is.

7. **8.2.12. Configure Compute Networking**

   Apply as is, except the following topics:

   a. **8.2.12.3. Configure the L2 Agent**

      Do **not** apply.

   b. **8.2.12.4. Configure Virtual Interface Plugging**

      Do **not** apply.

8. **8.2.13. Populate the Compute Service Database**

   Do **not** apply. Has been done on the Controller Node.

9. **8.2.14. Launch the Compute Services**

   a. **1. Starting the Message Bus Service**

      Apply as is.

   b. **2. Starting the Libvirtd Service**

      Apply as is.

   c. **3. Starting the API Service**

      Do **not** apply. Only required on the Controller Node.

   d. **4. Starting the Scheduler**

      Do **not** apply. Only required on the Controller Node.

   e. **5. Starting the Conductor**

Do **not** apply. Only required on the Controller Node.

f. **6. Starting the Compute Service**

   Apply as is.

10. **Additionally, perform the following steps**

   a. **Configure libvirt**

      Edit the `/etc/libvirt/qemu.conf` file to contain the following:

```
user = "root"
group = "root"

cgroup_device_acl = [
    "/dev/null", "/dev/full", "/dev/zero",
    "/dev/random", "/dev/urandom",
    "/dev/ptmx", "/dev/kvm", "/dev/kqemu",
    "/dev/rtc","/dev/hpet", "/dev/vfio/vfio",
    "/dev/net/tun"
]
```

   b. **Restart the libvirt service**

```
# systemctl restart libvirtd.service
```

   c. **Install nova-rootwrap network filters**

```
# yum install openstack-nova-network
# systemctl disable openstack-nova-network.service
```

   d. **Restart the Compute service**

```
# systemctl restart openstack-nova-compute.service
```

Networking Services (Neutron)

Controller Node

Important

Follow the Red Hat documentation's Chapter 7. Install OpenStack Networking instructions, but **note the following differences**.

1. **7.1. Install the OpenStack Networking Packages**

   Do **not** apply as is.

   Instead, install the following packages:

```
# yum install openstack-neutron python-networking-midonet-ext python-
neutronclient
# yum erase openstack-neutron-ml2
```

2. **7.2.1. Set the OpenStack Networking Plug-in**

   Do **not** apply. Instead, perform the following steps:

a. Edit the `/etc/neutron/neutron.conf` file and configure the following keys in the `[DEFAULT]` section:

```
[DEFAULT]
...
core_plugin = midonet.neutron.plugin_v2.MidonetPluginV2
service_plugins = midonet.neutron.services.l3.l3_midonet.
MidonetL3ServicePlugin
...
dhcp_agent_notification = False
...
allow_overlapping_ips = True
```

b. Create the directory for the MidoNet plugin:

```
mkdir /etc/neutron/plugins/midonet
```

c. Create the `/etc/neutron/plugins/midonet/midonet.ini` file and edit it to contain the following:

```
[DEFAULT]
...
core_plugin = midonet_v2_ext
service_plugins = midonet_l3_ext
...
dhcp_agent_notification = False
...
allow_overlapping_ips = True
...
rpc_backend = rabbit
...
auth_strategy = keystone
...
notify_nova_on_port_status_changes = True
notify_nova_on_port_data_changes = True
nova_url = http://controller:8774/v2.1
...
# In order to enable 'fip64' extension feature, the API extension path
 must
# be specified.  The path depends on the directory location in which
# python-networking-midonet-ext is installed.  For example, if the
 installation
# path is /usr/lib/python2.7/dist-packages/midonet-ext, add the
 following:
api_extensions_path = /usr/lib/python2.7/dist-packages/midonet-ext/
neutron/extensions

[database]
...
connection = mysql+pymysql://neutron:NEUTRON_DBPASS@controller/neutron

[oslo_messaging_rabbit]
...
rabbit_host = controller
rabbit_userid = openstack
rabbit_password = RABBIT_PASS

[keystone_authtoken]
...
auth_uri = http://controller:5000
auth_url = http://controller:35357
memcached_servers = controller:11211
auth_plugin = password
```

```
project_domain_id = default
user_domain_id = default
project_name = service
username = neutron
password = NEUTRON_PASS

[nova]
...
auth_url = http://controller:35357
auth_plugin = password
project_domain_id = default
user_domain_id = default
region_name = RegionOne
project_name = service
username = nova
password = NOVA_PASS

[oslo_concurrency]
...
lock_path = /var/lib/neutron/tmp
```

Note

When using multiple service plugins, separate them with commas:

```
[DEFAULT]
service_plugins = foo,bar,midonet.neutron.services.l3.
l3_midonet.MidonetL3ServicePlugin
```

3. **Configure the MidoNet plug-in**

   a. Create the directory for the MidoNet plugin:

   ```
   mkdir /etc/neutron/plugins/midonet
   ```

   b. Create the `/etc/neutron/plugins/midonet/midonet.ini` file and edit it to contain the following:

   ```
   [MIDONET]
   # MidoNet API URL
   midonet_uri = http://controller:8181/midonet-api
   # MidoNet administrative user in Keystone
   username = midonet
   password = MIDONET_PASS
   # MidoNet administrative user's tenant
   project_id = service
   # MidoNet API client
   client = midonet_ext.neutron.client.api.MidonetApiClient
   ```

   c. Create a symbolic link to direct Neutron to the MidoNet configuration:

   ```
   # ln -s /etc/neutron/plugins/midonet/midonet.ini /etc/neutron/plugin.
   ini
   ```

4. **7.2.2. Create the OpenStack Networking Database**

   Do **not** apply.

   Instead, create the database as follows:

```
$ mysql -u root -p
CREATE DATABASE neutron character set utf8;
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'localhost' IDENTIFIED BY
 'NEUTRON_DBPASS';
```

```
GRANT ALL PRIVILEGES ON neutron.* TO 'neutron'@'%' IDENTIFIED BY
 'NEUTRON_DBPASS';
FLUSH PRIVILEGES;
quit
```

Afterwards, run the `neutron-db-manage` command:

```
# su -s /bin/sh -c "neutron-db-manage --config-file /etc/neutron/
neutron.conf \
 --config-file /etc/neutron/plugins/midonet/midonet.ini upgrade head"
 neutron
# su -s /bin/sh -c "neutron-db-manage --subproject networking-midonet
 upgrade head" neutron
```

5. **7.2.3. Configure the OpenStack Networking Database Connection**

   Apply as is.

6. **7.2.4. Create the OpenStack Networking Identity Records**

   Apply as is.

7. **7.2.5. Configure OpenStack Networking Authentication**

   Apply as is.

8. **7.2.6. Configure the Firewall to Allow OpenStack Networking Traffic**

   Apply as is.

9. **7.2.7. Configure RabbitMQ Message Broker Settings for OpenStack Networking**

   Apply as is.

10. **7.2.8. Enable SSL Communication Between OpenStack Networking and the Message Broker**

    Apply as is.

11. **7.2.9. Configure OpenStack Networking to Communicate with the Compute Service**

    Apply as is.

12. **Configure Load-Balancer-as-a-Service (LBaaS)**

    Additionally to the Red Hat Installation Guide, configure Load-Balancer-as-a-Service (LBaaS) as described in the section called “Configure Load-Balancer-as-a-Service (LBaaS)” [14].

13. **Configure FireWall-as-a-Service (FWaaS)**

    Additionally to the Red Hat Installation Guide, configure FireWall-as-a-Service (FWaaS) as described in the section called “Configure FireWall-as-a-Service (FWaaS)” [15].

14. **7.2.10. Launch OpenStack Networking**

    Apply as is.

15. **7.3. Configure the DHCP Agent**

Do **not** apply.

167.4. Create an External Network

Do **not** apply.

Instead, create the Neutron networks after the OpenStack and MidoNet installation is completed.

Any networks that are created before the MidoNet plug-in is active will not be visible to MidoNet.

177.5. Configure the Plug-in Agent

Do **not** apply.

187.6. Configure the L3 Agent

Do **not** apply.

Configure Load-Balancer-as-a-Service (LBaaS)

1. **Install Neutron Load-Balancing-as-a-Service**

   ```
   # yum install python-neutron-lbaas
   ```

2. **Enable the MidoNet driver**

   Enable the MidoNet driver by using the `service_provider` option in the `/etc/neutron/neutron.conf` file:

   ```
   [service_providers]
   service_provider = LOADBALANCERV2:Midonet:midonet_ext.neutron.services.
   loadbalancer.v2_driver.MidonetLoadBalancerDriver:default
   ```

3. **Enable the LBaaS plug-in**

   Enable the LBaaS plug-in by using the `service_plugins` option in the `[DEFAULT]` section of the `/etc/neutron/neutron.conf` file:

   ```
   [DEFAULT]
   service_plugins = neutron_lbaas.services.loadbalancer.plugin.
   LoadBalancerPluginv2
   ```

   ### Note

   When using multiple service plugins, separate them with commas:

   ```
   [DEFAULT]
   service_plugins = foo,bar,lbassv2
   ```

4. **Enable load balancing in the dashboard**

   Follow the instructions in LBaaS Newton documentation.

5. **To finalize installation**

   Finalize the installation as described in Neutron Controller Node Installation.

Configure FireWall-as-a-Service (FWaaS)

1. **Install Neutron FireWall-as-a-Service**

   ```
   # yum install python-neutron-fwaas
   ```

2. **Enable the MidoNet FWaaS plug-in**

   Enable the MidoNet FWaaS plug-in by using the `service_plugins` option in the `/etc/neutron/neutron.conf` file:

   ```
   service_plugins = midonet.neutron.services.firewall.plugin.
   MidonetFirewallPlugin
   ```

   ### Note

   When using multiple service plugins, separate them with commas:

   ```
   [DEFAULT]
   service_plugins = foo,bar,midonet.neutron.services.firewall.
   plugin.MidonetFirewallPlugin
   ```

3. **Enable firewall in the dashboard**

   Change the `enable_firewall` option to `True` in the `/etc/openstack-dashboard/local_settings` file:

   ```
   OPENSTACK_NEUTRON_NETWORK = {
      'enable_firewall': True,
      ...
   }
   ```

4. **To finalize installation**

   Finalize the installation as described in Neutron Controller Node Installation.

Configure Quality of Service (QoS)

1. **Enable the QoS plug-in**

   Enable the QoS plug-in by using the `service_plugins` option in the `[DEFAULT]` section of the `/etc/neutron/neutron.conf` file:

   ```
   [DEFAULT]
   service_plugins = qos
   ```

   ### Note

   When using multiple service plugins, separate them with commas:

   ```
   [DEFAULT]
   service_plugins = foo,bar,qos
   ```

2. **Enable the MidoNet QoS notification driver**

   Enable the MidoNet QoS notification driver by using the `notification_drivers` option in the qos section of `/etc/neutron/neutron.conf` file:

   ```
   [qos]
   notification_drivers = midonet
   ```

3. **To finalize installation**

Finalize the installation as described in Neutron Controller Node Installation.

4. MidoNet Installation

Table of Contents

NSDB Nodes 17
Controller Node 20
Midolman Installation 22
MidoNet Host Registration 23

NSDB Nodes

ZooKeeper Installation

1. **Install ZooKeeper packages**

```
# yum install java-1.8.0-openjdk-headless
# yum install zookeeper zkdump nmap-ncat
```

2. **Configure ZooKeeper**

   a. **Common Configuration**

   Edit the `/etc/zookeeper/zoo.cfg` file to contain the following:

```
server.1=nsdb1:2888:3888
server.2=nsdb2:2888:3888
server.3=nsdb3:2888:3888
autopurge.snapRetainCount=10
autopurge.purgeInterval =12
```

   Create data directory:

```
# mkdir /var/lib/zookeeper/data
# chown zookeeper:zookeeper /var/lib/zookeeper/data
```

   Important

   For production deployments it is recommended to configure the storage of snapshots in a different disk than the commit log, this is done by setting the parameters `dataDir` and `dataLogDir` in `zoo.cfg`. In addition we advice to use an SSD drive for the commit log.

   b. **Node-specific Configuration**

   Important

   If using `CentOS 7.3` please use `/var/lib/zookeeper/myid` as the path for the host ID

   i. **NSDB Node 1**

   Create the `/var/lib/zookeeper/data/myid` file and edit it to contain the host's ID:

```
# echo 1 > /var/lib/zookeeper/data/myid
```

   ii. **NSDB Node 2**

   Create the `/var/lib/zookeeper/data/myid` file and edit it to contain the host's ID:

```
# echo 2 > /var/lib/zookeeper/data/myid
```

   iii. **NSDB Node 3**

   Create the `/var/lib/zookeeper/data/myid` file and edit it to contain the host's ID:

```
# echo 3 > /var/lib/zookeeper/data/myid
```

3. **Create Java Symlink**

```
# mkdir -p /usr/java/default/bin/
# ln -s /usr/lib/jvm/jre-1.8.0-openjdk/bin/java /usr/java/default/bin/
java
```

4. **Enable and start ZooKeeper**

```
# systemctl enable zookeeper.service
# systemctl start zookeeper.service
```

5. **Verify ZooKeeper Operation**

   After installation of all nodes has been completed, verify that ZooKeeper is operating properly.

   A basic check can be done by executing the `ruok` (Are you ok?) command on all nodes. This will reply with `imok` (I am ok.) if the server is running in a non-error state:

```
$ echo ruok | nc 127.0.0.1 2181
imok
```

   More detailed information can be requested with the `stat` command, which lists statistics about performance and connected clients:

```
$ echo stat | nc 127.0.0.1 2181
Zookeeper version: 3.4.5--1, built on 06/10/2013 17:26 GMT
Clients:
 /127.0.0.1:34768[0](queued=0,recved=1,sent=0)
 /192.0.2.1:49703[1](queued=0,recved=1053,sent=1053)

Latency min/avg/max: 0/4/255
Received: 1055
Sent: 1054
Connections: 2
Outstanding: 0
Zxid: 0x260000013d
Mode: follower
Node count: 3647
```

Cassandra Installation

1. **Install Cassandra packages**

```
# yum install java-1.8.0-openjdk-headless
# yum install dsc22
```

2. **Configure Cassandra**

a. **Common Configuration**

   Edit the `/etc/cassandra/conf/cassandra.yaml` file to contain the following:

```
# The name of the cluster.
cluster_name: 'midonet'

...

# Addresses of hosts that are deemed contact points.
seed_provider:
    - class_name: org.apache.cassandra.locator.SimpleSeedProvider
      parameters:
          - seeds: "nsdb1,nsdb2,nsdb3"
```

b. **Node-specific Configuration**

   i. **NSDB Node 1**

   Edit the `/etc/cassandra/conf/cassandra.yaml` file to contain the following:

```
# Address to bind to and tell other Cassandra nodes to connect to.
listen_address: nsdb1

...

# The address to bind the Thrift RPC service.
rpc_address: nsdb1
```

   ii. **NSDB Node 2**

   Edit the `/etc/cassandra/conf/cassandra.yaml` file to contain the following:

```
# Address to bind to and tell other Cassandra nodes to connect to.
listen_address: nsdb2

...

# The address to bind the Thrift RPC service.
rpc_address: nsdb2
```

   iii. **NSDB Node 3**

   Edit the `/etc/cassandra/conf/cassandra.yaml` file to contain the following:

```
# Address to bind to and tell other Cassandra nodes to connect to.
listen_address: nsdb3

...

# The address to bind the Thrift RPC service.
rpc_address: nsdb3
```

3. **Edit the service's init script**

   On installation, the `/var/run/cassandra` directory is created, but because it is located on a temporary file system it will be lost after system reboot. As a result it is not possible to stop or restart the Cassandra service anymore.

To avoid this, edit the `/etc/init.d/cassandra` file to create the directory on service start:

```
[...]
case "$1" in
    start)
        # Cassandra startup
        echo -n "Starting Cassandra: "
        mkdir -p /var/run/cassandra
        chown cassandra:cassandra /var/run/cassandra
        su $CASSANDRA_OWNR -c "$CASSANDRA_PROG -p $pid_file" > $log_file
 2>&1
        retval=$?
[...]
```

4. **Enable and start Cassandra**

```
# systemctl enable cassandra.service
# systemctl start cassandra.service
```

5. **Verify Cassandra Operation**

   After installation of all nodes has been completed, verify that Cassandra is operating properly.

   Important

   If Cassandra fails to start and prints a "buffer overflow" error message in its log file, you may try associating `127.0.0.1` with the hostname in `etc/hosts` (so that `hostname -i` will show `127.0.0.1`). This may solve the Cassandra start problem.

   A basic check can be done by executing the `nodetool status` command. This will reply with UN (Up / Normal) in the first column if the servers are running in a non-error state:

```
$ nodetool --host 127.0.0.1 status
[...]
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address    Load       Tokens  Owns   Host ID
       Rack
UN  192.0.2.1  123.45 KB  256     33.3%
 11111111-2222-3333-4444-555555555555  rack1
UN  192.0.2.2  234.56 KB  256     33.3%
 22222222-3333-4444-5555-666666666666  rack1
UN  192.0.2.3  345.67 KB  256     33.4%
 33333333-4444-5555-6666-777777777777  rack1
```

Controller Node

MidoNet Cluster Installation

1. **Install MidoNet Cluster package**

```
# yum install midonet-cluster
```

2. **Set up mn-conf**

   Edit `/etc/midonet/midonet.conf` to point mn-conf to the ZooKeeper cluster:

```
[zookeeper]
zookeeper_hosts = nsdb1:2181,nsdb2:2181,nsdb3:2181
```

3. **Configure access to the NSDB**

   This step needs to happen only once, it will set up access to the NSDB for the MidoNet Cluster and Agent nodes.

   Run the following command to set the cloud-wide values for the ZooKeeper and Cassandra server addresses:

```
$ cat << EOF | mn-conf set -t default
zookeeper {
    zookeeper_hosts = "nsdb1:2181,nsdb2:2181,nsdb3:2181"
}

cassandra {
    servers = "nsdb1,nsdb2,nsdb3"
}
EOF
```

   Run the following command to set the Cassandra replication factor:

```
$ echo "cassandra.replication_factor : 3" | mn-conf set -t default
```

4. **Configure Keystone access**

   This step needs to happen only once, it will set up access to Keystone for the MidoNet Cluster node(s).

   Determine `domain_name` and `domain_id` to be used for Keystone authentication:

```
# openstack domain list
+---------+---------+---------
+----------------------------------------------------------------------+
| ID      | Name    | Enabled | Description
                             |
+---------+---------+---------
+----------------------------------------------------------------------+
| default | Default | True    | Owns users and tenants (i.e. projects)
 available on Identity API v2. |
+---------+---------+---------
+----------------------------------------------------------------------+
```

   For `tenant_name`, use the project/tenant that the `midonet` user belongs to, as configured during user creation.

   Configure the authentication parameters for MidoNet Cluster via `mn-conf`:

```
$ cat << EOF | mn-conf set -t default
cluster.auth {
   admin_role = "admin"
   provider_class = "org.midonet.cluster.auth.keystone.KeystoneService"
   keystone {
      admin_token = ""
      protocol = "http"
      host = "controller"
      port = 35357
      domain_name = "Default"
      domain_id = "default"
      tenant_name = "$MIDONET_TENANT"
      user_name = "midonet"
      user_password = "$MIDONET_PASS"
      version = 3
   }
}
EOF
```

5. **Start the MidoNet Cluster**

```
# systemctl enable midonet-cluster.service
# systemctl start midonet-cluster.service
```

MidoNet CLI Installation

1. **Install MidoNet CLI package**

```
# yum install python-midonetclient
```

2. **Configure MidoNet CLI**

   Create the `~/.midonetrc` file and edit it to contain the following:

```
[cli]
api_url = http://controller:8181/midonet-api
username = MIDONET_USER
password = MIDONET_PASS
project_id = service
```

Midolman Installation

The *MidoNet Agent (Midolman)* has to be installed on all nodes where traffic enters or leaves the virtual topology, in this guide this are the **gateway1**, **gateway2** and **compute1** nodes.

1. **Install Midolman package**

```
# yum install java-1.8.0-openjdk-headless
# yum install midolman
```

2. **Set up mn-conf**

   Edit `/etc/midolman/midolman.conf` to point mn-conf to the ZooKeeper cluster:

```
[zookeeper]
zookeeper_hosts = nsdb1:2181,nsdb2:2181,nsdb3:2181
```

3. **Configure resource usage**

   Run these steps on **each agent host** in order to configure resource usage.

   Important

   For production environments the **large** templates are strongly recommended.

   a. **Midolman resource template**

      Run the following command to configure the Midolman resource template:

```
$ mn-conf template-set -h local -t TEMPLATE_NAME
```

      Replace **TEMPLATE_NAME** with one of the following templates:

```
agent-compute-large
```

```
agent-compute-medium
agent-gateway-large
agent-gateway-medium
default
```

b. **Java Virtual Machine (JVM) resource template**

   Replace the default `/etc/midolman/midolman-env.sh` file with one of the below to configure the JVM resource template:

```
/etc/midolman/midolman-env.sh.compute.large
/etc/midolman/midolman-env.sh.compute.medium
/etc/midolman/midolman-env.sh.gateway.large
/etc/midolman/midolman-env.sh.gateway.medium
```

4. **Configure MidoNet Metadata Proxy for all agents**

   This step needs to happen only once, it will set up MidoNet Metadata Proxy for all MidoNet Agent nodes.

   Run the following commands to set the cloud-wide values for the MidoNet Metadata Proxy:

```
$ echo "agent.openstack.metadata.nova_metadata_url : \"http:/
/controller:8775\"" | mn-conf set -t default
$ echo "agent.openstack.metadata.shared_secret : shared_secret" | mn-
conf set -t default
$ echo "agent.openstack.metadata.enabled : true" | mn-conf set -t
 default
```

   controller, **8775**, and **shared_secret** should be replaced with appropriate values. They need to match with the corresponding Nova Metadata API configuration.

   controller and **8775** specify the address on which Nova accepts Metadata API requests. **shared_secret** has to be the same as specified by the "metadata_proxy_shared_secret" field in the "neutron" section of nova.conf.

   The Nova side of the configuration for the metadata service is same as when using Neutron Metadata Proxy. See the OpenStack documentation for details:

   Cloud Administrator Guide: Configure Metadata

   ### Important

   The Metadata Proxy creates an interface on the hypervisor hosts, named "metadata".

   When using `iptables` it may be necessary to add a rule to accept traffic on that interface:

```
iptables -I INPUT 1 -i metadata -j ACCEPT
```

1. **Start Midolman**

```
# systemctl enable midolman.service
# systemctl start midolman.service
```

MidoNet Host Registration

1. **Launch MidoNet CLI**

```
$ midonet-cli
midonet>
```

2. **Create tunnel zone**

   MidoNet supports the Virtual Extensible LAN (VXLAN) and Generic Routing Encapsulation (GRE) protocols to communicate to other hosts within a tunnel zone.

   To use the VXLAN protocol, create the tunnel zone with type 'vxlan':

```
midonet> tunnel-zone create name tz type vxlan
tzone0
```

   To use the GRE protocol, create the tunnel zone with type 'gre':

```
midonet> tunnel-zone create name tz type gre
tzone0
```

Important

Make sure to allow GRE/VXLAN traffic for all hosts that belong to the tunnel zone. For VXLAN MidoNet uses UDP port 6677 as default.

1. **Add hosts to tunnel zone**

```
midonet> list tunnel-zone
tzone tzone0 name tz type vxlan

midonet> list host
host host0 name controller alive true
host host1 name gateway1 alive true
host host2 name gateway2 alive true
host host3 name compute1 alive true

midonet> tunnel-zone tzone0 add member host host0
 address ip_address_of_host0
zone tzone0 host host0 address ip_address_of_host0

midonet> tunnel-zone tzone0 add member host host1
 address ip_address_of_host1
zone tzone0 host host1 address ip_address_of_host1

midonet> tunnel-zone tzone0 add member host host2
 address ip_address_of_host2
zone tzone0 host host2 address ip_address_of_host2

midonet> tunnel-zone tzone0 add member host host3
 address ip_address_of_host3
zone tzone0 host host3 address ip_address_of_host3
```

5. Initial Network Configuration

Important

Follow the Red Hat documentation’s Create an external network instructions, but **note the following differences**.

1. **Creating and Configuring an External Network**

   Use the following command to create the external network:

   ```
   $ neutron net-create ext-net --router:external
   ```

6. Edge Router Setup

Prior to v5.0, with Neutron, you could set up the gateway in only one way, which was to have a special singleton gateway router called the Provider Router created implicitly when an external network was created in Neutron. The provider router sits at the edge of the cloud and interfaces with the uplink router. The Provider Router is where BGP was typically configured. The biggest limitation of this approach was that it took away the scenario in which you wanted to have an L2 network at the edge instead of a router. Another limitation was that only one such router could exist for the entire cloud.

These limitations are removed in v5.0, where you could design your gateway to be either L2 network or router with as many routers as you wish, all using the Neutron API.

There are two main changes:

Edge Router

The Provider Router is no longer implicitly created upon the external network creation. Instead, the edge gateway routers, called the Edge Routers, are created explicitly using standard Neutron API. With this approach, multiple Edge Routers can be created, and they are optional.

Gateway Virtual Topology

In the previous model, the Provider Router was connected directly to the tenant routers, with the external networks hanging off of the Provider Router.

In the new model, the external networks exist between the edge and the tenant routers.

To create the gateway topology issue the following Neutron commands.

Create a standard neutron router:

```
neutron router-create <EDGE_ROUTER_NAME>
```

Attach the edge router to an external network:

```
neutron router-interface-add <EDGE_ROUTER_ID> <EXT_SUBNET_ID>
```

Create a special network called `uplink` network, representing the physical network outside of the cloud:

```
neutron net-create <UPLINK_NET_NAME> --tenant_id admin --
provider:network_type uplink
```

Create a subnet for the uplink network matching the CIDR used in the uplink network (could just be /30 if linked directly to another router):

```
neutron subnet-create --tenant_id admin --disable-dhcp --name
 <UPLINK_SUBNET_NAME> <UPLINK_NET_NAME> <CIDR>
```

Create a port on the uplink network with a specific IP that you want to use and the binding details so that this virtual port gets bound to a specific NIC on the gateway host:

```
neutron port-create <UPLINK_NET_ID> --binding:host_id <HOST_NAME> --
binding:profile type=dict interface_name=<INTERFACE_NAME> --fixed-ip
 ip_address=<IP_ADDR>
```

Attach the uplink port to the Edge Router:

```
neutron router-interface-add <EDGE_ROUTER_ID> port=<UPLINK_PORT_ID>
```

7. BGP Uplink Configuration

MidoNet utilizes the Border Gateway Protocol (BGP) for external connectivity.

For production deployments it is strongly recommended to use BGP due to it’s scalability and redundancy.

For demo or POC environments, alternatively static routing can be used.

The following instructions assume below sample environment:

- One floating IP network
  - *192.0.2.0/24*
- Two MidoNet gateway nodes
  - *gateway1*, connecting to *bgp1* via *eth1*
  - *gateway2*, connecting to *bgp2* via *eth1*
- Two remote BGP peers
  - *bgp1*, *198.51.100.1*, AS *64513*
  - *bgp2*, *203.0.113.1*, AS *64514*
- Corresponding MidoNet BGP peers
  - *198.51.100.2*, AS *64512*
  - *203.0.113.2*, AS *64512*

Follow these steps to configure the BGP uplinks.

1. Launch the MidoNet CLI and find the Edge Router

   ```
   midonet-cli> router list
   router router0 name Edge Router state up
   router router1 name Tenant Router state up infilter chain0 outfilter
    chain1
   ```

   In this example the Edge Router is **router0**.

2. Create and bind virtual ports for the BGP sessions

   Refer to Chapter 6, “Edge Router Setup” [26] for instructions on how to create the necessary ports and bind them to the Gateway hosts' physical network interfaces.

   You can confirm the port configuration within MidoNet CLI by listing the Edge Router’s ports:

   ```
   midonet> router router0 port list
   port port0 device router0 state up mac fa:16:3e:11:11:11
    addresses 198.51.100.2/30
   port port1 device router0 state up mac fa:16:3e:22:22:22
    addresses 203.0.113.2/30
   [...]
   ```

3. Configure basic BGP settings

```
midonet> router router0 set asn 64512

midonet> router router0 add bgp-peer asn 64513 address 198.51.100.1
router0:peer0

midonet> router router0 add bgp-peer asn 64514 address 203.0.113.1
router0:peer1

midonet> router router0 list bgp-peer
peer peer0 asn 64513 address 198.51.100.1
peer peer1 asn 64514 address 203.0.113.1
```

4. If needed, configure MD5 authentication:

```
midonet> router router0 bgp-peer peer0 set password BGP_PASSWORD
midonet> router router0 bgp-peer peer1 set password BGP_PASSWORD
```

5. If needed, configure custom timers that will take precedence over the default ones defined in the MidoNet configuration:

```
midonet> router router0 bgp-peer peer0 set connect-retry 10
midonet> router router0 bgp-peer peer0 set hold-time 5
midonet> router router0 bgp-peer peer0 set keep-alive 5
midonet> router router0 bgp-peer peer1 set connect-retry 10
midonet> router router0 bgp-peer peer1 set hold-time 5
midonet> router router0 bgp-peer peer1 set keep-alive 5
midonet> router router0 list bgp-peer
peer peer0 asn 64513 address 198.51.100.1 keep-alive 5 hold-time 5
 connect-retry 10
peer peer1 asn 64514 address 203.0.113.1 keep-alive 5 hold-time 5
 connect-retry 10
```

6. Add routes to the remote BGP peers

   In order to be able to establish connections to the remote BGP peers, corresponding routes have to be added.

```
midonet> router router0 route add src 0.0.0.0/0 dst 198.51.100.0/30
 port router0:port0 type normal
router0:route0

midonet> router router0 route add src 0.0.0.0/0 dst 203.0.113.0/30
 port router0:port1 type normal
router0:route1
```

7. Advertise BGP routes

   In order to provide external connectivity for hosted virtual machines, the floating IP network has to be advertised to the BGP peers.

```
midonet> router router0 add bgp-network net 192.0.2.0/24
router0:net0

midonet> router router0 list bgp-network
net net0 net 192.0.2.0/24
```

8. Further Steps

MidoNet installation and integration into OpenStack is completed.

Note

Consult the **Operations Guide** for further instructions on operating MidoNet.